MACHINE LEARNING · MODEL EVALUATION

Regression & Classification Metrics Zero to Pro

A complete, from-scratch guide to measuring how good your ML models actually are — the math, the geometric intuition, the pitfalls, and the scikit-learn code — explained in plain language with visuals.

MAE · MSE · RMSE | R^2 & Adjusted R^2 | Accuracy & Confusion Matrix | Precision · Recall · F1 | Multi-Class Averaging

Dark-mode edition · Built for comfortable reading

Table of Contents

Part 1 — Regression Metrics ···

1.1 Why do we need metrics at all?

1.2 Mean Absolute Error (MAE)

1.3 Mean Squared Error (MSE)

1.4 Root Mean Squared Error (RMSE)

1.5 R^2 Score (Coefficient of Determination)

1.6 Adjusted R^2 Score

1.7 scikit-learn code walkthrough

Part 2 — Classification Metrics ···

2.1 Accuracy Score

2.2 Why accuracy can lie — imbalanced data

2.3 The Confusion Matrix (TP, TN, FP, FN)

2.4 Type 1 vs Type 2 Errors

2.5 Multi-class confusion matrices

1.1 Why Do We Need Metrics At All?

Suppose you fit a regression line to some data — CGPA on the x-axis, placement package (in LPA, "Lakhs Per Annum") on the y-axis. The line is drawn. Great. But now what? **How good is that line?** Did the model actually learn something useful, or is it guessing?

This is exactly what regression metrics answer. They take your **actual values** (y) and your **model's predicted values** ($\hat{y}$, read "y-hat") and compress the gap between them into a single, comparable number.

The 5 metrics we'll build from scratch:

1. **MAE** — Mean Absolute Error
2. **MSE** — Mean Squared Error
3. **RMSE** — Root Mean Squared Error
4. **R²** — R-Squared / Coefficient of Determination
5. **Adjusted R²** — a corrected version of R²

No single metric is "the best" — each dataset has its own quirks (outliers, number of features, units), and different metrics behave differently in each situation. That's why all five exist side by side.

The core idea behind every regression metric

For every single data point *i*, we look at the **residual** — the gap between what actually happened and what the model predicted:

RESIDUAL FOR POINT I

$$\text{residual}_i = y_i - \hat{y}_i$$

Every metric below is just a different, clever way of turning this list of *n* residuals into one meaningful number.

PART 1 · REGRESSION METRICS

1.2 Mean Absolute Error (MAE)

The simplest, most intuitive regression metric — "on average, how far off is my model?"

Basic idea

For every data point, measure the distance between the actual value and the predicted value. Ignore direction (a point above the line is just as wrong as one below it) — so we take the **absolute value**. Then average all these distances across every point.

FORMULA

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$$

WORKED EXAMPLE

ACTUAL Y	PREDICTED Ŷ	|Y − Ŷ|
3.89	3.49	0.40
4.10	3.89	0.21
3.00	3.30	0.30

MAE = (0.40 + 0.21 + 0.30) / 3 = **0.30** — meaning the model is, on average, off by **0.30 LPA**.

Geometric intuition

Picture the regression line running through your scatter plot. For every point, draw a straight vertical segment connecting it to the line. MAE is simply the **average length** of all those segments.

PART 1 · REGRESSION METRICS

1.3 Mean Squared Error (MSE)

Same idea as MAE, but square the errors instead of taking their absolute value.

Basic idea

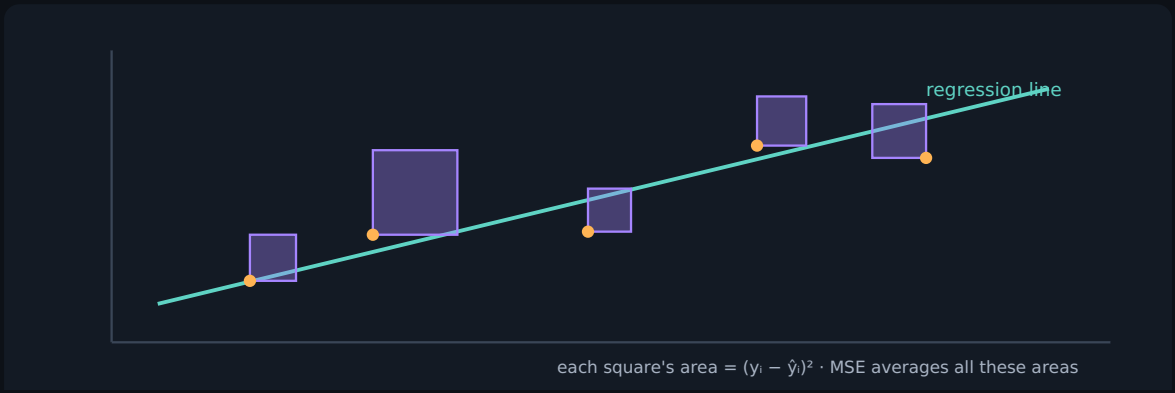
Take the exact same residuals as before, but instead of $|y_i - \hat{y}_i|$, square them: $(y_i - \hat{y}_i)^2$. Squaring automatically removes the sign issue (no need for absolute value) *and* gives us a perfectly smooth, differentiable curve — which is exactly why MSE is the default loss function used inside Linear Regression itself.

FORMULA

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Geometric intuition — literal squares

This is the fun part. If you draw an actual square whose side length equals the residual for each point, MSE is the process of **summing the areas of all these squares** and then averaging them. Minimizing MSE literally means minimizing total squared area.



PART 1 · REGRESSION METRICS

1.4 Root Mean Squared Error (RMSE)

Exactly MSE — just with the units fixed.

RMSE solves MSE's biggest annoyance (the squared unit) by simply taking the square root at the end.

FORMULA

$$\text{RMSE} = \sqrt{\left(\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2\right)}$$

PROPERTY	MSE	RMSE
Differentiable	Yes	Yes
Unit	LPA² (squared, hard to read)	LPA (same as y, easy to read)
Robust to outliers	No	No
Typical usage	Loss function internals	Reporting / Deep Learning

PRACTICAL ADVICE

As a beginner, don't overthink which single metric to pick — **calculate MAE, MSE, and RMSE together** for every model you build. Only in deep learning, where you must optimize toward one specific loss, does the choice get trickier (reducing one metric can sometimes increase another).

PART 1 · REGRESSION METRICS

1.5 R² Score (Coefficient of Determination)

Also called "Goodness of Fit" — the only metric so far that is context-independent.

The problem MAE / MSE / RMSE don't solve

Is an MAE of 1.5 "good"? Depends entirely on context — 1.5 LPA might be fine, but 1.5 degrees of steering-angle error in a self-driving car is catastrophic. R² fixes this: a score like **0.72** means the same thing regardless of the dataset.

Building the intuition — the "worst possible model"

Imagine you had *no* input column (no CGPA) — only past package values. The best you could do for any new student is predict the **mean** of all past packages, $\bar{y}$. This "mean line" is your worst-case baseline.

Now compare that baseline to your actual regression line. R² measures: *how much better is my regression line than just guessing the mean every time?*

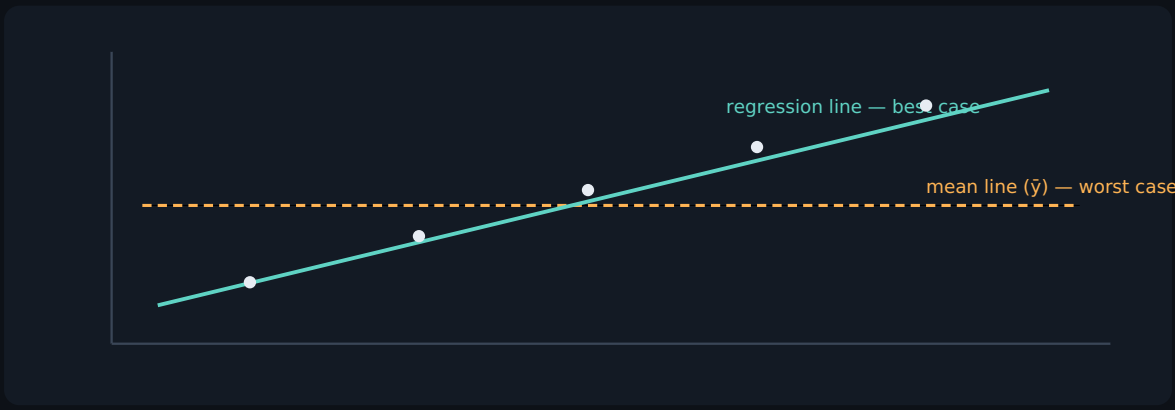


Figure 1.3 — R² compares regression-line error against mean-line error.

FORMULA

$$R^2 = 1 - \frac{SSR}{}$$

PART 1 · REGRESSION METRICS

1.6 Adjusted R² Score

Fixing R²'s one big flaw: it never wants to go down.

The flaw

Every time you add **any** new input column to your model — even a completely irrelevant one, like the *temperature* on interview day — plain R² will either **increase slightly or stay the same.** It never decreases. This makes R² unreliable for deciding whether a new feature is actually useful.

FORMULA

$$\text{Adjusted } R^2 = 1 - \frac{(1 - R^2)(n - 1)}{n - 1 - k}$$

n = number of rows · k = number of independent (input) columns

Why this works — two cases

CASE A — ADD AN IRRELEVANT COLUMN

k increases → denominator $(n - 1 - k)$ shrinks. R² barely moves, so $(1 - R^2)$ stays roughly constant. Smaller denominator + same numerator = **bigger fraction** → after "1 −", the final score **goes down**.

CASE B — ADD A GENUINELY USEFUL COLUMN

k increases (denominator shrinks a bit) but R² **jumps up noticeably**, so $(1 - R^2)$ shrinks fast — faster than the denominator shrinks. The whole fraction gets smaller → the final score **goes up**.

WORKED COMPARISON — SAME 165-ROW PLACEMENT DATASET

MODEL COLUMNS	R²	ADJUSTED R²	VERDICT

1.7 Putting It All Together in scikit-learn

Same 165-row CGPA → Package placement dataset used throughout this section.

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=2)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# --- MAE ---
mae = mean_absolute_error(y_test, y_pred)          # 0.28  (LPA)

# --- MSE ---
mse = mean_squared_error(y_test, y_pred)           # 0.12  (LPA^2)

# --- RMSE (no direct function, wrap in sqrt) ---
rmse = np.sqrt(mse)                                # 0.34  (LPA)

# --- R2 score ---
r2 = r2_score(y_test, y_pred)                      # 0.78

# --- Adjusted R2 (compute manually) ---
n = X_test.shape[0]
k = X_test.shape[1]                                # number of input columns
adj_r2 = 1 - ((1-r2)*(n-1)) / (n-1-k)   # 0.77
```

Reading these numbers together:

- MAE = 0.28 → on average the model is off by 0.28 LPA

PART 2 · CLASSIFICATION METRICS

2.1 Accuracy Score

Classification metrics answer a similar question as regression metrics, but for problems where the output is a **category** (placed / not placed, disease / no disease) instead of a number. Accuracy is the simplest place to start.

Basic idea

For every prediction, check: did the model get it right or wrong? Then:

FORMULA

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

WORKED EXAMPLE — 10 STUDENTS

#	ACTUAL	LOGISTIC REGRESSION	DECISION TREE
1	Placed	**Placed ✓**	**Placed ✓**
2	Not Placed	**Placed ✗**	**Not Placed ✓**
3	Not Placed	**Not Placed ✓**	**Not Placed ✓**
...	...	...	...

Out of 10 predictions, Logistic Regression got **8 right → 80% accuracy**. The Decision Tree got **9 right → 90% accuracy** — so on this sample, the Decision Tree wins.

Multi-class works exactly the same way

2.2 Why Accuracy Can Lie — Imbalanced Data

Accuracy has a deep flaw: it tells you *how many* mistakes happened, but never *what kind.* There are always two possible mistakes in binary classification:

- **Mistake type 1:** actually positive, but model said negative
- **Mistake type 2:** actually negative, but model said positive

Accuracy blends both into one number — which becomes dangerously misleading on **imbalanced datasets** (one class vastly outnumbers the other).

THE AIRPORT SECURITY EXAMPLE

Say you're building a model that scans passenger photos to flag potential terrorists. Out of 100,000 passengers, maybe **only 1** is actually a threat. Now build the laziest possible "model": *always predict "not a terrorist," no matter what.*

This model will be wrong exactly **once** (on the one real terrorist) and right for all 99,999 others.

$$\text{Accuracy} = \frac{99{,}999}{100{,}000} = 99.99\%$$

A model that **never catches a single terrorist** scores 99.99% accuracy — clearly, accuracy alone is the wrong tool here. This is exactly the scenario where we need **Precision** and **Recall** — but first we need the tool that reveals the *type* of mistake: the **Confusion Matrix**.

2.3 The Confusion Matrix

The confusion matrix is a grid that shows exactly how many predictions fell into each combination of (actual, predicted). In scikit-learn, **predicted values sit on the columns** and **actual values sit on the rows.**

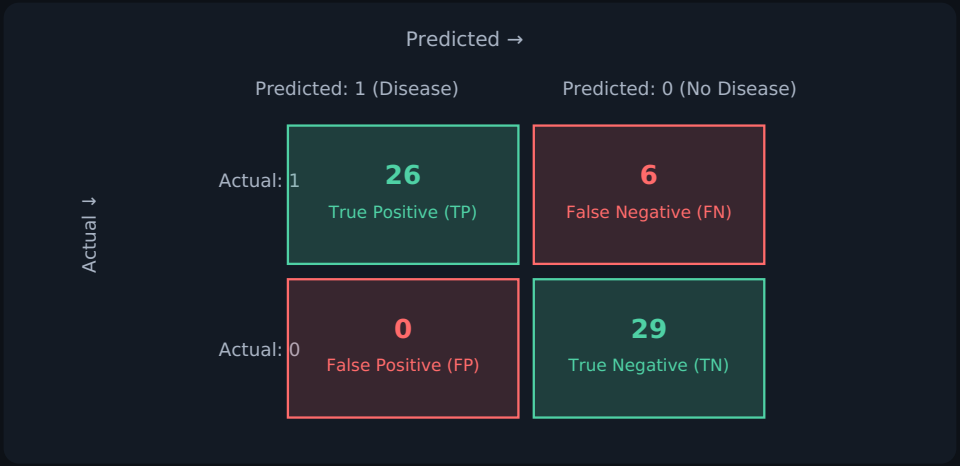


Figure 2.1 — Heart-disease confusion matrix (Logistic Regression). Correct predictions sit on the diagonal.

The naming "cheat code"

Every cell name has two words: **[True/False] + [Positive/Negative]**

- The **second word** (Positive / Negative) = what the **model predicted**.
- The **first word** (True / False) = whether that prediction was **correct**.

MODEL PREDICTED	ACTUALLY WAS	CELL NAME

2.4 Type 1 vs Type 2 Errors

TYPE 1 ERROR = FALSE POSITIVE

Model said **Positive (1)**, but the truth was **Negative (0)**.

Example: predicting a healthy patient has heart disease.

TYPE 2 ERROR = FALSE NEGATIVE

Model said **Negative (0)**, but the truth was **Positive (1)**.

Example: telling a sick patient they're disease-free.

MEMORY HOOK

False **P**ositive = Type **1** · False **N**egative = Type **2**. This exact terminology is a favorite interview question.

2.5 Multi-Class Confusion Matrices

With *c* classes, the confusion matrix becomes a **c × c** grid instead of 2×2. The logic doesn't change — diagonal = correct, everything off-diagonal = a specific type of mistake.

IRIS DATASET — 3 CLASSES (0 = SETOSA, 1 = VERSICOLOR, 2 = VIRGINICA)

	PRED: 0	PRED: 1	PRED: 2
Actual: 0	**7**	0	0
Actual: 1	0	**9**	2
Actual: 2	0	0	**21**

Diagonal (7, 9, 21) = correct predictions. The "2" in row "Actual:1" means 2 true Versicolor flowers were mistakenly predicted as

2.6 Precision

Precision answers: "*Out of everything my model called positive, how many actually were?*"

FORMULA

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

MANAGER SCENARIO — GMAIL SPAM FILTER

Two models, A and B, both have **80% accuracy** on 1000 emails. So accuracy can't tell them apart. Look deeper:

MODEL	TP	FP	FN
A	100	30	lower
B	100	10	higher

False Positive here = a **real, important email wrongly marked as spam** — e.g. your job offer letter, buried and lost. That's the more dangerous mistake for this problem, so we want **fewer False Positives → higher Precision.**

$$\text{Precision}_A = 100 / (100+30) = 0.769 \quad | \quad \text{Precision}_B = 100 / (100+10) = 0.909$$

Model B wins — deploy B.

WHEN TO OPTIMIZE FOR PRECISION

Whenever a **False Positive is the costlier mistake** — e.g. flagging legitimate email as spam, falsely accusing someone,

2.7 Recall

Recall answers the mirror-image question: "*Out of everyone who is actually positive, how many did my model catch?*"

FORMULA

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

MANAGER SCENARIO — CANCER DETECTION FROM X-RAYS

Two models, both 90% accurate. Compare False Negatives (a cancer patient told they're healthy):

MODEL	TP	FN
A	1000	200 (lower)
B	1000	500 (higher)

$$\text{Recall}_A = 1000/(1000+200) = 0.833 \quad | \quad \text{Recall}_B = 1000/(1000+500) = 0.667$$

A False Negative here means a **sick patient sent home undiagnosed** — potentially fatal. **Model A**, with higher Recall, is the safer choice to deploy.

WHEN TO OPTIMIZE FOR RECALL

Whenever a **False Negative is the costlier mistake** — disease screening, fraud detection, safety-critical alarms.

2.8 F1 Score

Precision and Recall trade off against each other — pushing one up usually pulls the other down. F1 combines both into a single number using the **harmonic mean** (not a simple average).

FORMULA

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why harmonic mean, not arithmetic mean?

The harmonic mean **punishes extreme low values** much harder than an arithmetic mean does.

EXTREME CASE

Precision = 0, Recall = 100.

- Arithmetic mean: (0 + 100) / 2 = **50** — misleadingly decent-looking.
- Harmonic mean (F1): **0** — correctly reflects that the model is useless (0 precision means literally nothing it labels positive is trustworthy).

MILDER CASE

MODEL	PRECISION	RECALL	ARITHMETIC MEAN	F1 (HARMONIC)
A	80	80	80	80
B	60	100	80	75

2.9 Multi-Class Precision, Recall & F1

In binary classification, scikit-learn silently focuses on the "1" class and ignores "0" by default (you can force both with `average=None`). With **3+ classes**, you must decide how to combine per-class scores into one number.

WORKED EXAMPLE — DOG / CAT / RABBIT CLASSIFIER

CLASS	CORRECTLY PREDICTED	TOTAL PREDICTED AS THIS CLASS	ACTUAL COUNT	PRECISION	RECALL
Dog	25	29	30	25/29 = 0.86	25/30 = 0.83
Cat	30	45	34	30/45 = 0.66	30/34 = 0.88
Rabbit	20	34	34	20/34 = 0.58	20/34 = 0.58

Two ways to combine per-class scores

Macro average — plain, unweighted mean across classes.

$$\text{Macro Precision} = (0.86+0.66+0.58)/3 = \mathbf{0.70}$$

Use when classes are roughly balanced in size.

Weighted average — each class's score is weighted by how many real examples it has.

$$= (0.86 \cdot 30 + 0.66 \cdot 34 + 0.58 \cdot 34) / 98 = \mathbf{0.70}$$

Use when classes are imbalanced.

F1 for multi-class follows the same two-step recipe: compute F1 *per class* from that class's own Precision & Recall, then take the Macro

2.10 classification_report — Everything At Once

Rather than calling precision, recall and F1 separately for every class, scikit-learn bundles it all into one function.

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

TYPICAL OUTPUT (ILLUSTRATIVE)

CLASS	PRECISION	RECALL	F1-SCORE	SUPPORT
0	0.90	0.85	0.87	60
1	0.83	0.88	0.85	55
accuracy	*0.86*	*115*		
macro avg	0.865	0.865	0.86	115
weighted avg	0.866	0.86	0.86	115

Support = the actual number of true instances of that class in the test set.

TAKEAWAY

`classification_report` is usually the very first thing to run after any classification model — it hands you accuracy, per-class precision/recall/F1, and both averaging strategies in one shot.

Cheat Sheet — All Formulas In One Place

Regression metrics

MAE

$$(1/n)\ \Sigma\ |y_i - \hat{y}_i|$$

Same unit as y · robust-ish to outliers · not differentiable at 0

MSE

$$(1/n)\ \Sigma\ (y_i - \hat{y}_i)^2$$

Differentiable (used as loss) · squared units · sensitive to outliers

RMSE

$$\sqrt{\text{MSE}}$$

Same unit as y · still sensitive to outliers

R^2

$$1 - \text{SSR/SSM}$$

Context-free · 1=perfect, 0=useless, <0=worse than mean

Adjusted R^2

$$1 - [(1-R^2)(n-1)]/(n-1-k)$$

Penalizes irrelevant featu · use with many input columns

Classification metrics

Accuracy

$$(TP+TN)/(TP+TN+FP+FN)$$

Misleading on imbalanced data

Precision

$$TP/(TP+FP)$$

Use when False Positives are costly

Recall

$$TP/(TP+FN)$$

Use when False Negatives are costly

F1 Score

$$2\cdot(P\cdot R)/(P+R)$$

Harmonic mean — use when neither error is clearly worse

Type 1 error = False Positive · **Type 2 error** = False Negative
Macro average = unweighted mean across classes · **Weighted average** = weighted by class support

End of guide — go apply these on a real dataset and calculate every metric yourself. That's the fastest way it truly sticks.